

```
=====
=====
FILE: src/App.tsx
=====
=====
```

```
import './index.css';
```

```
import type { ReactElement } from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';
```

```
import { ThemeProvider } from './context/ThemeContext';
import { AuthProvider } from './context/AuthContext';
import { useAuth } from './hooks/useAuth'
```

```
';  
;  
import Navbar from './Components/Navbar'  
;  
import Home from './pages/Home';  
import Dashboard from './pages/Dashboard'  
;  
import Trade from './pages/Trade';  
import Stake from './pages/Stake';  
import WalletDashboard from './pages/Wal  
letDashboard';  
import Leaderboard from './pages/Leaderb  
oard';  
import Account from './pages/Account';  
import Onboarding from './pages/Onboardi  
ng';
```

```
const ProtectedRoute = ({ children }: {  
  children: ReactElement }) => {  
  const { user, loading } = useAuth();  
  if (loading) {
```

```

    return <div className="px-8 py-6">Ch
checking session...</div>;
  }
  if (!user) {
    return <Navigate to="/Account" repla
ce />;
  }
  return children;
};

```

```

function App() {
  return (
    <ThemeProvider>
      <AuthProvider>
        <>
          <Navbar />
          <main className="mt-10">
            <Routes>
              <Route path="/" element={<
Home />} />

```

```
<Route
  path="/Dashboard"
  element={
    <ProtectedRoute>
      <Dashboard />
    </ProtectedRoute>
  }
/>
```

```
<Route
  path="/Trade"
  element={
    <ProtectedRoute>
      <Trade />
    </ProtectedRoute>
  }
/>
```

```
<Route
  path="/Stake"
  element={
    <ProtectedRoute>
```

```
        <Stake />
      </ProtectedRoute>
    }
  />
  <Route
    path="/Wallet-Dashboard"
    element={
      <ProtectedRoute>
        <WalletDashboard />
      </ProtectedRoute>
    }
  />
  <Route
    path="/Leaderboard"
    element={
      <ProtectedRoute>
        <Leaderboard />
      </ProtectedRoute>
    }
  />
```

```

        <Route
            path = "/Onboarding"
            element = {
                <ProtectedRoute>
                    <Onboarding/>
                </ProtectedRoute>
            }
        />
        <Route path="/Account" ele
ment={<Account />} />
        <Route path="*" element={<
Navigate to="/" replace />} />
    </Routes>
    </main>
</>
</AuthProvider>
</ThemeProvider>
);
}

```

```
export default App;
```

```
=====
```

```
=====
```

```
FILE: src/Components/Button.tsx
```

```
=====
```

```
=====
```

```
import type buttonProps from '../types/  
Button';  
//Retrieve button props from the buttonP  
rops interface
```

```
const Button: React.FC<buttonProps> = ({  
children, variant, className, size, onCl
```

```
    onClick, type, disabled}) => {  
      //Creates a forward component so that the children can be passed like a regular button: <Button> text</Button>.
```

```
      //Receives button props - text, variant, className, size
```

```
      return (
```

```
        <button onClick = {onClick} className={`btn btn-${variant} btn-${size} ${className}`} type = {type} disabled={disabled}>
```

```
          {children}  
        </button>
```

```
      );
```

```
};
```

```
export default Button;
```



```
=====
=====
FILE: src/Components/CryptoChart.tsx
=====
=====
```

```
import { useState } from "react";
import SearchBar from "../SearchBar";
import { TradingViewChart } from "../TradingChart";
import { coinNameToTicker } from "../data/tickers";
import { useFetchCoinMetrics } from "../utils/FetchCoinMetrics";
```

```
const CryptoChart = () => {
  const [coin, setCoin] = useState("bitcoin");
```

```
const handleSearch = (value: string) =  
> {  
    setCoin(value.toLowerCase());  
};
```

```
const logoToken = import.meta.env.VITE  
_LOGO_API;  
const ticker = coinNameToTicker[coin];
```

```
const { data, loading, error } = useFe  
tchCoinMetrics(coin);
```

```
// --- ERROR STATE: Unknown Coin Ticke  
r ---  
if (!ticker) {  
    return (  
        <div className="p-8 text-center bg  
-surface min-h-screen text-primary">  
            <SearchBar Search={handleSearch}
```

```

/>
    <p className="mt-8 text-xl text-
danger">
        ⚠ Unknown cryptocurrency: <st
rong>{coin}</strong>
    </p>
    <p className="text-muted">Try a
common coin like "bitcoin" or "ethereum"
.</p>
</div>
);
}

// --- LOADING/ERROR STATES ---
if (loading) {
    return (
        <div className="p-8 text-center bg
-surface min-h-screen text-primary">
            <SearchBar Search={handleSearch}
/>

```

```
        <div className="mt-20 text-xl animate-pulse">Loading data for {coin}...</div>
```

```
    </div>
```

```
  );
```

```
}
```

```
if (error || !data) {
```

```
  return (
```

```
    <div className="p-8 text-center bg-surface min-h-screen text-primary">
```

```
      <SearchBar Search={handleSearch}
```

```
    />
```

```
    <div className="mt-20 text-xl text-danger">
```

```
      ✖ Fetch failed: {error || 'No data returned from API.'}
```

```
    </div>
```

```
  </div>
```

```
);
```

```
}
```

```
// --- RENDER DASHBOARD ---
```

```
// Determine color for 24h change (Success/Danger is common in trading UI)
const changeColor = data.change24h >=
0 ? 'text-success' : 'text-danger';
const changeSign = data.change24h >= 0
? '↑' : '↓';
```

```
return (
  <div className="min-h-screen bg-surface text-primary p-4 sm:p-8 font-sans">
    <div className="max-w-7xl mx-auto"
  >
```

```
    {/* 1. Search Bar (Top Control)
```

```
- Assuming SearchBar has minimal styling
  */}
```

```

        <div className="mb-8">
            <SearchBar Search={handleSearch}
h} />
        </div>

        {/* 2. Header and Price Metrics
*/}

        <div className="flex items-center
r mb-6 space-x-4 border-b border-divider
pb-4">
            <img
                src={`https://img.logokit.co
m/crypto/${ticker}?token=${logoToken}`}
                alt={`${coin} logo`}
                className="w-10 h-10 rounded
-full shadow-md"
            />
            <div>
                <h1 className="text-2xl font
-semibold capitalize">{coin}</h1>

```

```
        <p className="text-sm text-m
uted">{ticker} / USD</p>
    </div>
```

```
    {/* Current Price and 24h Chan
ge */}
```

```
        <div className="ml-auto flex i
tems-end space-x-4">
```

```
            <p className="text-4xl font-
bold">${data.price.toLocaleString('en-US
', { minimumFractionDigits: 2, maximumFr
actionDigits: 8 })}</p>
```

```
            <p className={`text-lg font-
medium ${changeColor} mb-0.5`}>
```

```
                {changeSign} {data.change2
4h.toFixed(2)}%
```

```
            </p>
```

```
        </div>
```

```
    </div>
```

```

        {/* 3. Main Chart (Placeholder)
*/}
        <div className="bg-card rounded-
lg p-2 shadow-lg h-[450px] mb-8 border b
order-divider">
            <TradingViewChart coin={coin}
mode="candlestick" />
        </div>

        {/* 4. Key Metrics Grid */}
        <div className="grid grid-cols-2
md:grid-cols-4 gap-4">
            {/* Market Cap */}
            <MetricCard
                title="Market Cap"
                value={`$$${(data.marketCap /
1e9).toFixed(2)}B`}
                tooltip={`$$${data.marketCap.
toLocaleString()}`}
            />

```



```

        { /* 24h Volume */
        <MetricCard
            title="24h Volume"
            value={`$$${(data.volume24h /
1e9).toFixed(2)}B`}
            tooltip={`$$${data.volume24h.
toLocaleString()}`}
        />

```

```

        { /* Fully Diluted Valuation (F
DV) */
        <MetricCard
            title="FDV"
            value={data.fullyDilutedValu
ation
? `$$${(data.fullyDilutedVa
luation / 1e9).toFixed(2)}B`
: 'N/A'
        }

```

```

        tooltip={data.fullyDilutedVal
uation
        ? `${data.fullyDilutedVal
uation.toLocaleString()}`
        : 'Not available'
    }
/>

```

```

    {/* Circulating Supply */}
    <MetricCard
        title="Circulating Supply"
        value={`$${(data.circulatingS
upply / 1e6).toFixed(2)}M`}
        tooltip={data.circulatingSup
ply.toLocaleString()}
    />
</div>

```

```

    {/* 5. Additional Supply Data (M
uted Footer) */}

```

```
        <div className="mt-8 text-xs text-muted text-right">
```

```
            Total Supply: {data.totalSupply?.toLocaleString() || 'Unlimited/N/A'}
        </div>
```

```
    </div>
```

```
</div>
```

```
);
};
```

```
// Helper component for clean metric display
```

```
const MetricCard = ({ title, value, tooltip }: { title: string, value: string, tooltip: string }) => (
```

```
    <div className="bg-card p-4 rounded-md shadow-sm border border-divider hover:shadow-lg transition duration-150 relativ
```

```

e group">
    <p className="text-sm font-medium text-muted">{title}</p>
    <p className="text-xl font-semibold mt-1">{value}</p>
    {/* Tooltip implementation remains the same for functionality */}
    <span className="absolute bottom-full left-1/2 transform -translate-x-1/2 mb-2 px-3 py-1 bg-tooltip text-xs text-tooltip-text rounded opacity-0 group-hover:opacity-100 transition-opacity duration-300 pointer-events-none whitespace-nowrap">
        {tooltip}
    </span>
</div>
);

export default CryptoChart;

```

```
=====
=====
FILE: src/Components/DMLMToggle.tsx
=====
=====
```

```
import { useTheme } from "../hooks/useTh
eme";
import Button from "../Button";
export const Toggle = () => {
  const {isDark, toggleTheme} = useTheme
();
  return (
    <Button
      onClick={toggleTheme}
      size="lg"
```

```
    type="button"  
    variant="secondary"
```

```
>
```

```
    {isDark ? <img className = "w-5 h-  
5" src = "/public/sun.svg"/> : <img clas  
sName = "w-5 h-5" src = "/public/moon.sv  
g"/>}
```

```
  </Button>
```

```
);  
};
```

```
=====
```

```
=====
```

```
FILE: src/Components/Navbar.tsx
```

```
=====
```

```
=====
```

```
import Button from "../Button";  
import { useNavigate, useLocation } from
```

```
"react-router-dom";  
import { Links } from "../lib/links";  
import { Toggle } from "../DMLMToggle";
```

```
const Navbar = () => {  
  const navigate = useNavigate();  
  const location = useLocation();  
  const {pathname} = location;  
  return (  

```

```
<nav className="bg-neutral-primary w-full z-20 top-0 start-0 ">
```

```
  <div className="max-w-7xl flex flex-wrap items-center justify-between mx-auto p-4">
```

```
    <a href="/" className="flex items-center space-x-3 rtl:space-x-reverse">
```

```
      <span className="self-center text
```

```
t-xl text-heading font-semibold whitespa  
ce-nowrap">Block Finance</span>  
</a>
```

```
<button data-collapse-toggle="navbar  
-default" type="button" className="inlin  
e-flex items-center p-2 w-10 h-10 justif  
y-center text-sm text-body rounded-base  
md:hidden hover:bg-neutral-secondary-sof  
t hover:text-heading focus:outline-none  
focus:ring-2 focus:ring-neutral-tertiary  
" aria-controls="navbar-default" aria-ex  
panded="false">
```

```
<span className="sr-only">Open m  
ain menu</span>
```

```
<svg className="w-6 h-6" aria-hi  
dden="true" xmlns="http://www.w3.org/200  
0/svg" width="24" height="24" fill="none  
" viewBox="0 0 24 24"><path stroke="curr  
entColor" strokeLinecap="round" strokeWi  
dth="2" d="M5 7h14M5 12h14M5 17h14"/></s
```



```

vg>
    </button>
    <div className="hidden w-full md:blo
ck md:w-auto" id="navbar-default">
        <ul className="font-medium flex flex
-col p-4 md:p-0 mt-4 border rounded-lg m
d:flex-row md:space-x-8 md:mt-0 md:bord
er-0">

            {Links.map((item) => (
                <li key={item.link}>
                    <Button
                        onClick={() => navigate(item.lin
k)}
                        type="button"
                        variant="secondary"
                        size="md"
                        //Created a template string that
checked whether the pathname is equal t
o the link and changed the color of the

```

text accordingly.

```
      className={` ${pathname === item
.link ? 'text-gray-800' : ''} `}
    >
      {item.name}
    </Button>
```

```
  </li>
```

```
  ))}
```

```
<Toggle/>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
</nav>
```

```
);
```

```
};
```

```
export default Navbar
```

```
=====
=====
FILE: src/Components/NewsCard.tsx
=====
=====
```

```
interface NewsCardProps {
  title: string;
  description: string | null;
  link: string;
  imageUrl: string | null;
  pubDate: string;
  source: string;
}
```

```
const NewsCard = ({ title, description,
```

```
link, imageUrl, pubDate, source }: NewsC  
ardProps) => {  
  const formatDate = (dateString: string  
) => {  
    const date = new Date(dateString);  
    const now = new Date();  
    //Difference in hours  
    const diffInHours = Math.floor((now.  
getTime() - date.getTime()) / (1000 * 60  
* 60));  
    //Use if statements to determine how  
recent a post is for the user - improvi  
ng user experience  
    if (diffInHours < 1) return 'Just no  
w';  
    if (diffInHours < 24) return `${diff  
InHours}h ago`;  
    const diffInDays = Math.floor(diffIn  
Hours / 24);  
    if (diffInDays < 7) return `${diffIn
```

```
Days}d ago`;
    return date.toLocaleDateString('en-US', { month: 'short', day: 'numeric' });
};
```

```
return (
    <a
      href={link}
      target="_blank"
      rel="noopener noreferrer"
      className="
        block rounded-xl border border-[va
r(--border-color)]
        bg-[var(--surface-color)] text-[va
r(--text-color)]
        no-underline overflow-hidden hover
:shadow-md
        transition-shadow duration-200 gro
up
        "
```

>

```
<div className="w-full h-40 overf
low-hidden ">
  <img
    //If image url isn't present then
    use the backup image
    src={imageUrl || "/backup.webp"
  }
    alt={title}
    //Typical tailwindcss animation
    for image upscale on hover
    className="w-full h-full object
-cover group-hover:scale-105 transition-
transform duration-200"
  />
</div>

<div className="p-4">
```

```

        <div className="flex items-center justify-between mb-2">
            <span className="text-xs font-medium uppercase text-[var(--muted-text-color)]">
                {source}
            </span>
            <span className="text-xs text-[var(--muted-text-color)]">
                {formatDate(pubDate)}
            </span>
        </div>
        <h3 className="text-sm font-semibold mb-2 line-clamp-2">
            {title}
        </h3>
        {/*If description then showcase it */}
        {description && (
            <p className="text-xs text-[va

```

```
    r(--muted-text-color)] line-clamp-2">
        {description}
    </p>
  )}
</div>
</a>
);
};
```

```
export default NewsCard;
```

```
=====
=====
FILE: src/Components/SearchBar.tsx
=====
=====
```

```
import { useState } from 'react';
```



```
import Button from './Button';
```

```
interface SearchBarProps {  
  Search: (coin: string) => void;  
}
```

```
const SearchBar: React.FC<SearchBarProps  
> = ({ Search }) => {  
  const [query, setQuery] = useState("")  
;
```

```
  const handleClick = () => {  
    Search(query.trim());  
  };
```

```
  return (
```

```
    <>
```

```
    <form
```

```
      className="max-w-md mx-auto"
```

```
    onSubmit={e => {
      e.preventDefault(); // prevents
page reload
      handleClick();
    }}
  >
```

```
    <label htmlFor="search" className=
"block mb-2.5 text-sm font-medium text-h
eading sr-only">
      Search
    </label>
```

```
    <div className="relative">
      <div className="absolute inset-y
-0 start-0 flex items-center ps-3 pointe
r-events-none">
        <svg
          className="w-4 h-4 text-body
"
          aria-hidden="true"
```

```

0/svg"
    xmlns="http://www.w3.org/200
width="24"
height="24"
fill="none"
viewBox="0 0 24 24"
>
    <path stroke="currentColor"
strokeLinecap="round" strokeWidth="2" d=
"m21 21-3.5-3.5M17 10a7 7 0 1 1-14 0 7 7
0 0 1 14 0Z" />
    </svg>
</div>

<input
  type="search"
  value={query}
  onChange={(e) => setQuery(e.ta
rget.value)}
  className="block w-full p-3 ps

```

```
-9 bg-neutral-secondary-medium text-head-
ing text-sm rounded-base shadow-xs pla-
ceholder:text-body"
    placeholder="Search"
  />
```

```
    <Button
      variant = "secondary"
      size = "sm"
      type="submit"
      className="absolute end-0 bott-
om-0 border border-transparent focus:
ring-4 focus:ring-brand-medium shadow-xs
font-medium leading-5 rounded text-xs p-
x-3 py-1.5"
    >
      Search
    </Button>
  </div>
</form></>
```

```
    );  
};
```

```
export default SearchBar;
```

```
=====
```

```
=====
```

```
FILE: src/Components/Tooltip.tsx
```

```
=====
```

```
=====
```

```
import { HelpCircle } from 'lucide-react'  
;  
import { useState, type ReactNode } from  
  'react';
```

```
interface TooltipProps {
```

```
  text: string;  
  children?: ReactNode;  
  position?: 'top' | 'bottom';  
}
```

```
const Tooltip = ({ text, children, position = 'top' }: TooltipProps) => {  
  const [open, setOpen] = useState(false);  
  const tooltipPositionClass = position === 'top' ? 'bottom-full mb-2' : 'top-full mt-2';
```

```
  const accessibleLabel =  
    typeof children === 'string' && children.trim() !== '' ? `More info about ${children}` : 'More info';
```

```
  return (  
    <span className="relative inline-block"
```

```
x align-baseline">
  <span
    role="button"
    tabIndex={0}
    className="inline-flex items-bas
eline cursor-help border-b border-dashed
border-current bg-transparent p-0"
    onMouseEnter={() => setOpen(true
)}
    onMouseLeave={() => setOpen(fals
e)}
    onFocus={() => setOpen(true)}
    onBlur={() => setOpen(false)}
    onClick={() => setOpen((value) =
> !value)}
    onKeyDown={(event) => {
      if (event.key === 'Enter' || e
vent.key === ' ') {
        event.preventDefault();
        setOpen((value) => !value);
```

```

    }
  }}
  style={{ lineHeight: 'inherit' }
}

aria-label={accessibleLabel}
aria-expanded={open}
>
  <span className="inline">{children}</span>
    <HelpCircle className="ml-1 inline-block h-3 w-3" style={{ verticalAlign: 'baseline', position: 'relative', top: '0.05em' }} />
  </span>

{open && (
  <span
    role="tooltip"
    className={`pointer-events-none absolute left-1/2 z-[70] w-max max-w-x

```



```

s -translate-x-1/2 rounded-lg border bor
der-[var(--border-color)] bg-[var(--surf
ace-color)] px-3 py-2 text-xs shadow-xl
${tooltipPositionClass}`}
        style={{ color: 'var(--text-co
lor)' }}
    >
        {text}
    </span>
)}
</span>
);
};

```

```

export default Tooltip;

```

```

=====
=====

```

FILE: src/Components/TradingChart.tsx

=====

=====

```
import {
  createChart,
  ColorType,
  CandlestickSeries,
  LineSeries,
  type CandlestickData,
  type IChartApi,
  type ISeriesApi,
  type UTCTimestamp,
} from 'lightweight-charts';
import { useEffect, useRef, useState } from 'react';
import { fetchCandleData } from '../utils/FetchCandleData';
import { useTheme } from '../hooks/useTheme';
```

```
type ChartMode = 'line' | 'candlestick';

interface TradingViewChartProps {
  coin: string;
  mode?: ChartMode;
  onMarketDataChange?: (marketData: {
    lastPrice: number | null;
    change24h: number | null;
  }) => void;
}
```

```
const TradingViewChart = ({
  coin,
  mode = 'candlestick',
  onMarketDataChange,
}: TradingViewChartProps) => {
  const { isDark } = useTheme();
  const chartContainerRef = useRef<HTMLD
  ivElement | null>(null);
```

```
const chartRef = useRef<IChartApi | null>(null);
const candleSeriesRef = useRef<ISeriesApi<'Candlestick'> | null>(null);
const lineSeriesRef = useRef<ISeriesApi<'Line'> | null>(null);
const [candles, setCandles] = useState<CandlestickData<UTCTimestamp>[]>([]);
```

```
useEffect(() => {
  if (!chartContainerRef.current) {
    return;
  }

```

```
const chart = createChart(chartContainerRef.current, {
  layout: {
    background: { type: ColorType.Solid, color: isDark ? '#000000' : '#ffffff' },
  },

```

```
        textColor: isDark ? '#e5e7eb' :  
'#1f1f1f',  
    },  
    grid: {  
        vertLines: { color: isDark ? '#3  
4343a' : '#e5e7eb' },  
        horzLines: { color: isDark ? '#3  
4343a' : '#e5e7eb' },  
    },  
    width: chartContainerRef.current.c  
lientWidth,  
    height: 400,  
});
```

```
    if (mode === 'candlestick') {  
        candleSeriesRef.current = chart.ad  
dSeries(CandlestickSeries, {  
            upColor: '#26a69a',  
            downColor: '#ef5350',  
            wickUpColor: '#26a69a',
```

```
        wickDownColor: '#ef5350',
        borderVisible: false,
    });
    lineSeriesRef.current = null;
} else {
    lineSeriesRef.current = chart.addSeries(
        LineSeries, {
            color: '#2962FF',
            lineWidth: 2,
        });
    candleSeriesRef.current = null;
}
```

```
chartRef.current = chart;
```

```
const handleResize = () => {
    if (chartContainerRef.current) {
        chart.applyOptions({ width: chartContainerRef.current.clientWidth });
    }
}
```

```
};
```

```
    window.addEventListener('resize', handleResize);
```

```
    return () => {  
        window.removeEventListener('resize', handleResize);  
        chart.remove();  
        chartRef.current = null;  
        candleSeriesRef.current = null;  
        lineSeriesRef.current = null;  
    };  
}, [isDark, mode]);
```

```
useEffect(() => {  
    let mounted = true;
```

```
    const loadCandles = async () => {  
        setCandles([]);
```

```
    const nextCandles = await fetchCandleData(coin);  
    if (mounted) {  
        setCandles(nextCandles);  
    }  
};
```

```
void loadCandles();
```

```
return () => {  
    mounted = false;  
};  
}, [coin]);
```

```
useEffect(() => {  
    if (candles.length === 0) {  
        return;  
    }  
}
```

```
if (mode === 'candlestick' && candle
```



```
SeriesRef.current) {  
    candleSeriesRef.current.setData(candles);  
    chartRef.current?.timeScale().fitContent();  
    return;  
}
```

```
    if (mode === 'line' && lineSeriesRef.current) {  
        const lineData = candles.map((candle) => ({  
            time: candle.time,  
            value: candle.close,  
        }));  
        lineSeriesRef.current.setData(lineData);  
        chartRef.current?.timeScale().fitContent();  
    }
```

```
}, [candles, mode]));
```

```
const latestChartPrice = candles.length > 0 ? candles[candles.length - 1].close : null;
```

```
const price24hAgo = candles.length > 24 ? candles[candles.length - 25].close : null;
```

```
const chartChange24h = latestChartPrice !== null && price24hAgo !== null && Number.isFinite(price24hAgo) && price24hAgo > 0 ? ((latestChartPrice - price24hAgo) / price24hAgo) * 100 : null;
```

```
useEffect(() => {  
  if (!onMarketDataChange) {  
    return;
```

```
}
```

```
onMarketDataChange({  
  lastPrice: latestChartPrice,  
  change24h: chartChange24h,  
});
```

```
}, [chartChange24h, latestChartPrice,  
onMarketDataChange]);
```

```
return <div ref={chartContainerRef} st  
yle={{ width: '100%', height: '400px' }}  
>;  
};
```

```
export type { ChartMode };  
export { TradingViewChart };
```

```
=====
```

=====

FILE: src/context/AuthContext.tsx

=====

=====

```
import {
  useEffect,
  useState,
  type ReactNode,
} from 'react';
import { supabase } from '../lib/supabaseClient';
import { AuthContext, type AuthContextType } from '../authContextStore';

export const AuthProvider = ({ children
}: { children: ReactNode }) => {
  const [user, setUser] = useState<AuthContextType['user']>(null);
  const [loading, setLoading] = useState
```

```
(true);
```

```
useEffect(() => {  
  const init = async () => {  
    const { data, error } = await supabase.auth.getUser();  
    if (!error && data.user) {  
      setUser(data.user);  
    }  
    setLoading(false);  
  };  
};
```

```
void init();
```

```
const {  
  data: { subscription },  
} = supabase.auth.onAuthStateChange(  
  (_event, session) => {  
    setUser(session?.user ?? null);  
  });
```

```
return () => {  
    subscription.unsubscribe();  
};  
}, []);
```

```
//
```